BÁO CÁO BÀI TẬP THỰC HÀNH LAB 01

BÀI 1:

BÀI 1:

Generate

```
import torch

a = torch.tensor(
    [[1, 2, 3],
     [4, 5, 6]],
    dtype=torch.int32
)
print(a)
```

```
tensor([[1, 2, 3],
        [4, 5, 6]], dtype=torch.int32)
```

Yêu cầu: Cho biết kích thước, số chiều và kiểu dữ liệu của Tensor a.

```
print("Kích thước:", a.shape)
print("Số chiều:", a.ndim)
print("Kiểu dữ liệu:", a.dtype)
```

```
Kích thước: torch.Size([2, 3])
Số chiều: 2
Kiểu dữ liệu: torch.int32
```

BÀI 2:

BÀI 2:

```python
import torch

a = torch.rand(3, 4)
b = torch.randn(3, 4)
c = torch.zeros(2, 3)
d = torch.ones(2, 3)
e = torch.eye(4)

print("Tensor a (rand):\n", a)
print("\nTensor b (randn):\n", b)
print("\nTensor c (zeros):\n", c)
print("\nTensor d (ones):\n", d)
print("\nTensor e (eye):\n", e)
```

```
Tensor a (rand):
 tensor([[0.8225, 0.6875, 0.8706, 0.7909],
        [0.5585, 0.0283, 0.5630, 0.2507],
        [0.1113, 0.3671, 0.4508, 0.6678]])

Tensor b (randn):
 tensor([[ 0.5921, -0.2612, -0.1331,  1.5533],
        [-0.8877, -0.4135,  0.8972,  0.0971],
        [ 0.1949,  0.4557, -0.3613, -0.2447]])

Tensor c (zeros):
 tensor([[0., 0., 0.],
        [0., 0., 0.]])

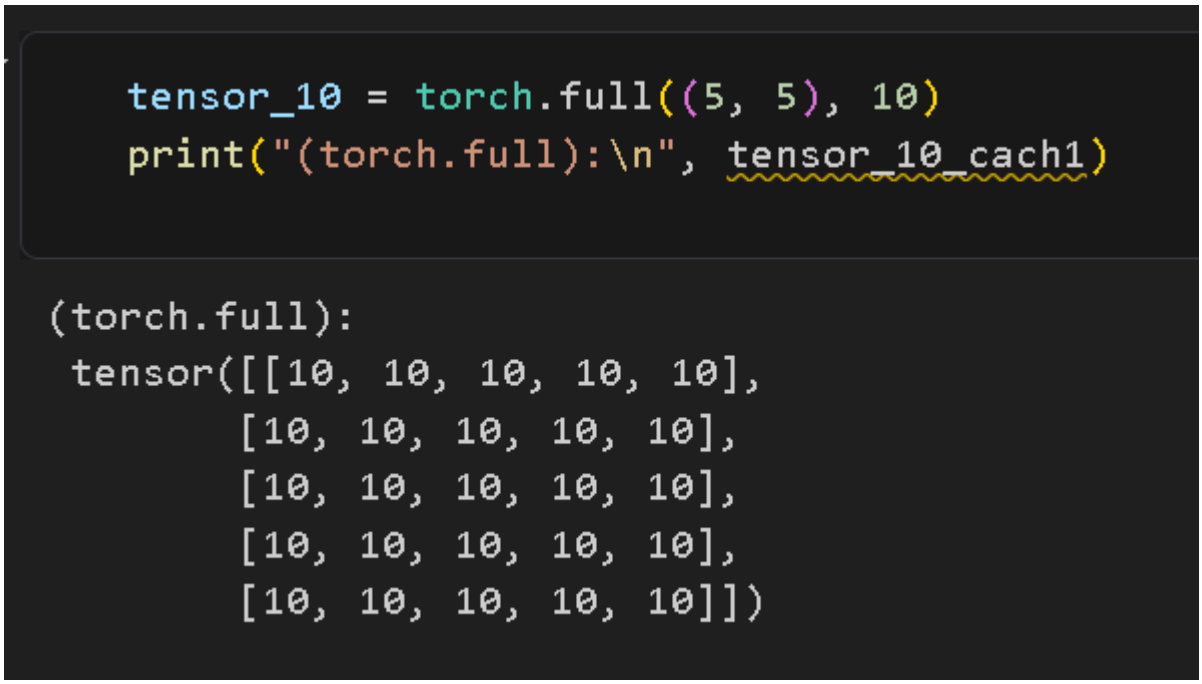
Tensor d (ones):
 tensor([[1., 1., 1.],
        [1., 1., 1.]])

Tensor e (eye):
 tensor([[1., 0., 0., 0.],
        [0., 1., 0., 0.],
        [0., 0., 1., 0.],
        [0., 0., 0., 1.]])
```

Yêu cầu 1: Nhận xét sự khác nhau giữa `torch.rand()` và `torch.randn()`.

- `torch.rand()`: Tạo giá trị ngẫu nhiên phân phối đều từ 0 đến 1 (chỉ có số dương).
- `torch.randn()`: Tạo giá trị ngẫu nhiên phân phối chuẩn (bao gồm cả số âm và dương).

Yêu cầu 2: Tạo thêm một Tensor 5x5 chứa toàn giá trị 10.



```
    tensor_10 = torch.full((5, 5), 10)
    print("(torch.full):\n", tensor_10_cach1)
```

```
(torch.full):
 tensor([[10, 10, 10, 10, 10],
         [10, 10, 10, 10, 10],
         [10, 10, 10, 10, 10],
         [10, 10, 10, 10, 10],
         [10, 10, 10, 10, 10]])
```

BÀI 3:

BÀI 3:

```python
import torch

a = torch.zeros(2, 3, 4)

print("Tensor:\n", a)
print("Shape:", a.shape)
print("Size:", a.size())
print("Number of dimensions:", a.ndim)
print("Data type:", a.dtype)
```

```
Tensor a:
 tensor([[[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]],

        [[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]]])
Shape: torch.Size([2, 3, 4])
Size: torch.Size([2, 3, 4])
Number of dimensions: 3
Data type: torch.float32
```

Yêu cầu: Giải thích ý nghĩa của `torch.Size([2, 3, 4])`.

- Giải thích: Tensor này có 3 chiều, bao gồm 2 ma trận (khối) riêng biệt, mỗi ma trận có 3 hàng và 4 cột.

BÀI 4:

BÀI 4:

```python
import torch

a = torch.randn(3, 4, 5)
print("Original tensor:")
print(a)

print("\nTensor a[1]:")
print(a[1])

print("\nTensor a[1:, 2:4]:")
print(a[1:, 2:4])

print(a[1].shape)
print(a[1:, 2:4].shape)
```

```
Original tensor:
tensor([[[ 0.5734,  0.4808, -0.1358, -1.1762,  0.3477],
         [-1.0945,  0.2698,  0.5049, -0.6115,  0.1069],
         [ 1.0481,  0.8514, -0.0673, -1.1741, -0.5916],
         [-1.6707,  0.7466,  0.0340,  0.5446, -1.7583]],

        [[ 0.9103,  1.2440,  1.5391, -0.6584,  1.1386],
         [ 0.6309,  0.6083, -0.8772,  0.7950, -1.3331],
         [-0.1403, -0.1342, -0.1924,  0.6660, -0.2985],
         [ 0.7241, -0.3246, -0.2163,  0.0375,  0.7349]],

        [[-0.9194, -0.4536, -1.9005, -1.2151,  1.0922],
         [-0.9073, -0.3287,  0.4341, -0.0884,  1.2380],
         [ 0.4022, -1.6952,  0.1253,  0.5111,  1.2400],
         [-0.7053,  2.2384,  0.6741, -0.6633,  1.3055]]])
```

```
Tensor a[1]:
tensor([[ 0.9103,  1.2440,  1.5391, -0.6584,  1.1386],
        [ 0.6309,  0.6083, -0.8772,  0.7950, -1.3331],
        [-0.1403, -0.1342, -0.1924,  0.6660, -0.2985],
        [ 0.7241, -0.3246, -0.2163,  0.0375,  0.7349]])

Tensor a[1:, 2:4]:
tensor([[[-0.1403, -0.1342, -0.1924,  0.6660, -0.2985],
         [ 0.7241, -0.3246, -0.2163,  0.0375,  0.7349]],
...
        [[ 0.4022, -1.6952,  0.1253,  0.5111,  1.2400],
         [-0.7053,  2.2384,  0.6741, -0.6633,  1.3055]]])
torch.Size([4, 5])
torch.Size([2, 2, 5])
```

```
    hai_phan_tu_cuoi = a[:, :, -2:]
    print("Hai phần tử cuối cùng trên chiều thứ ba:\n", hai_phan_tu_cuoi)
```

```
Hai phần tử cuối cùng trên chiều thứ ba:
 tensor([[[-1.1762,  0.3477],
         [-0.6115,  0.1069],
         [-1.1741, -0.5916],
         [ 0.5446, -1.7583]],

        [[-0.6584,  1.1386],
         [ 0.7950, -1.3331],
         [ 0.6660, -0.2985],
         [ 0.0375,  0.7349]],

        [[-1.2151,  1.0922],
         [-0.0884,  1.2380],
         [ 0.5111,  1.2400],
         [-0.6633,  1.3055]]])
```

BÀI 5:

BÀI 5:

```
import torch

a = torch.randn(3, 4)
print("Original:")
print(a)

print("\nFlatten:")
print(a.ravel())

print("\nReshape to 3x2x2:")
print(a.reshape(3, 2, 2))
```

```
Original:
tensor([[ 0.6730, -0.2914,  0.0378,  1.3375],
        [-0.6617,  0.2103,  1.3938, -1.6948],
        [-1.7204, -0.4307,  1.3493, -0.8951]])

Flatten:
tensor([ 0.6730, -0.2914,  0.0378,  1.3375, -0.6617,  0.2103,  1.3938, -1.6948,
        -1.7204, -0.4307,  1.3493, -0.8951])

Reshape to 3x2x2:
tensor([[[ 0.6730, -0.2914],
         [ 0.0378,  1.3375]],

        [[-0.6617,  0.2103],
         [ 1.3938, -1.6948]],

        [[-1.7204, -0.4307],
         [ 1.3493, -0.8951]]])
```

```python
print("Số phần tử của Tensor gốc (3x4):", a.numel())

a_reshaped = a.reshape(3, 2, 2)
print("Số phần tử của Tensor sau khi reshape (3x2x2):", a_reshaped.numel())
```

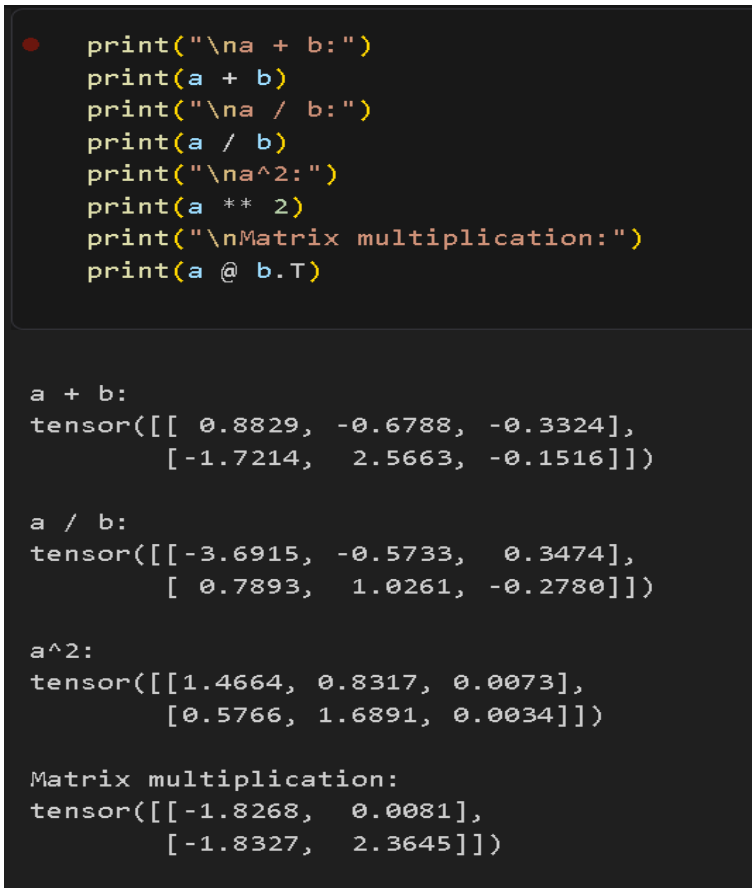
```
Số phần tử của Tensor gốc (3x4): 12
Số phần tử của Tensor sau khi reshape (3x2x2): 12
```

```python
a_2x6 = a.reshape(2, 6)
print("\nReshape to 2x6:\n", a_2x6)
print("New shape:", a_2x6.shape)
```

```
Reshape to 2x6:
 tensor([[ 0.6730, -0.2914,  0.0378,  1.3375, -0.6617,  0.2103],
        [ 1.3938, -1.6948, -1.7204, -0.4307,  1.3493, -0.8951]])
New shape: torch.Size([2, 6])
```

BÀI 6:


BÀI 6:
import torch
a = torch.randn(2, 3)
b = torch.randn(2, 3)
print("a:")
print(a)
print("b:")
print(b)
a:
tensor([[1.2110, 0.9120, -0.0857],
[-0.7594, 1.2997, 0.0584]])
b:
tensor([[-0.3280, -1.5907, -0.2467],
[-0.9620, 1.2666, -0.2100]])


print("\na + b:")
print(a + b)
print("\na / b:")
print(a / b)
print("\na^2:")
print(a ** 2)
print("\nMatrix multiplication:")
print(a @ b.T)
a + b:
tensor([[0.8829, -0.6788, -0.3324],
[-1.7214, 2.5663, -0.1516]])
a / b:
tensor([[-3.6915, -0.5733, 0.3474],
[0.7893, 1.0261, -0.2780]])
a^2:
tensor([[1.4664, 0.8317, 0.0073],
[0.5766, 1.6891, 0.0034]])
Matrix multiplication:
tensor([[-1.8268, 0.0081],
[-1.8327, 2.3645]])

Yêu cầu: Phân biệt phép toán a * b với a @ b.T

Phân biệt: a * b là nhân từng phần tử tương ứng của hai ma trận cùng kích thước. a @ b.T là phép nhân ma trận đại số tuyến tính (hàng nhân cột), yêu cầu phải chuyển vị ma trận b.

```
    ket_qua = a @ b.T
    print("Shape của kết quả a @ b.T là:", ket_qua.shape)
```

```
Shape của kết quả a @ b.T là: torch.Size([2, 2])
```

BÀI 7:

```
    import torch
    a = torch.randn(3, 4)
    print(a)
    print("\nMean:")
    print(torch.mean(a, dim=0))
    print("\nStandard deviation:")
    print(torch.std(a, dim=0))
    print("\nCumulative sum:")
    print(torch.cumsum(a, dim=0))
```

```
tensor([[-0.6346, -0.1030,  1.1493,  1.1042],
        [-0.6405,  0.7976,  0.8172,  0.1223],
        [ 0.2852,  0.3204, -0.0727,  1.2672]])

Mean:
tensor([-0.3299,  0.3383,  0.6313,  0.8312])

Standard deviation:
tensor([0.5328, 0.4506, 0.6318, 0.6194])

Cumulative sum:
tensor([[-0.6346, -0.1030,  1.1493,  1.1042],
        [-1.2750,  0.6945,  1.9665,  1.2264],
        [-0.9898,  1.0149,  1.8939,  2.4936]])
```

```python
print("\nMean (dim=1):")
print(torch.mean(a, dim=1))

print("\nStandard deviation (dim=1):")
print(torch.std(a, dim=1))

print("\nCumulative sum (dim=1):\n")
print(torch.cumsum(a, dim=1))
```

```
Mean (dim=1):
tensor([ 0.1781, -0.5555,  0.0395])

Standard deviation (dim=1):
tensor([0.8946, 0.7272, 1.6478])

Cumulative sum (dim=1):

tensor([[-1.0917, -0.7417, -0.2942,  0.7125],
        [-0.9358, -0.5779, -1.8824, -2.2220],
        [-0.5865, -2.3059, -2.0268,  0.1582]])
```

Nhận xét: Khi dùng `dim=1`, các phép tính trung bình, độ lệch chuẩn và cộng dồn được thực hiện theo chiều ngang (từng hàng) thay vì từng cột.

BÀI 8:

```python
import torch
x = torch.tensor(
3.6,
requires_grad=True
)
y = x * x
y.backward()
print("x =", x)
print("y =", y)
print("x.grad =", x.grad)
```

```
x = tensor(3.6000, requires_grad=True)
y = tensor(12.9600, grad_fn=<MulBackward0>)
x.grad = tensor(7.2000)
```

```python
import torch
x_new = torch.tensor(3.6, requires_grad=True)
y_new = 3*x_new**2 + 2*x_new + 1
y_new.backward()
print("Hàm số mới: y = 3x^2 + 2x + 1 tại x = 3.6")
print("Giá trị đạo hàm tính bởi PyTorch (x_new.grad):", x_new.grad)
```

```
Hàm số mới: y = 3x^2 + 2x + 1 tại x = 3.6
Giá trị đạo hàm tính bởi PyTorch (x_new.grad): tensor(23.6000)
```

BÀI 9:

```python
import numpy as np
import torch
polynomial = np.poly1d([1, 2, 3])
N = 20
X = np.random.randn(N, 1) * 5
Y = polynomial(X)
XX = np.hstack([
    X * X,
    X,
np.ones_like(X)
])
w = torch.randn(3, 1, requires_grad=True)
x = torch.tensor(XX, dtype=torch.float32)
y = torch.tensor(Y, dtype=torch.float32)
optimizer = torch.optim.NAdam([w], lr=0.01)
print("Initial coefficients:")
print(w)
for epoch in range(1000):
    optimizer.zero_grad()
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    mse.backward()
    optimizer.step()
print("Learned coefficients:")
print(w)
```

```
Initial coefficients:
tensor([[ 0.6584],
        [-0.2786],
        [-0.1531]], requires_grad=True)
Learned coefficients:
tensor([[1.0021],
        [1.9992],
        [2.8352]], requires_grad=True)
```

Yêu cầu: So sánh ba hệ số học được với [1, 2, 3] và giải thích vai trò của các hàm.

- So sánh: Các hệ số học được xấp xỉ rất sát với đa thức gốc [1, 2, 3].
- Giải thích: `optimizer.zero_grad()` xóa gradient của vòng lặp cũ; `mse.backward()` tính đạo hàm sai số; `optimizer.step()` cập nhật lại trọng số dựa trên đạo hàm vừa tính.

BÀI 10:

```python
import random
import torch
A = torch.tensor(random.random(), requires_grad=True)
B = torch.tensor(random.random(), requires_grad=True)
C = torch.tensor(random.random(), requires_grad=True)
D = torch.tensor(random.random(), requires_grad=True)
EPOCHS = 2000
optimizer = torch.optim.NAdam([A, B, C, D], lr=0.01)
for _ in range(EPOCHS):
    y1 = A + B - 9
    y2 = C - D - 1
    y3 = A + C - 8
    y4 = B - D - 2
    sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4
    optimizer.zero_grad()
    sqerr.backward()
    optimizer.step()

print("A =", A)
print("B =", B)
print("C =", C)
print("D =", D)
```

```
A = tensor(4.7386, requires_grad=True)
B = tensor(4.2613, requires_grad=True)
C = tensor(3.2613, requires_grad=True)
D = tensor(2.2613, requires_grad=True)
```

```
    print("\n--- Kiểm tra lại nghiệm ---")
    print(f"A + B = {A.item() + B.item():.4f} (Kỳ vọng: 9)")
    print(f"C - D = {C.item() - D.item():.4f} (Kỳ vọng: 1)")
    print(f"A + C = {A.item() + C.item():.4f} (Kỳ vọng: 8)")
    print(f"B - D = {B.item() - D.item():.4f} (Kỳ vọng: 2)")
```

```
--- Kiểm tra lại nghiệm ---
A + B = 8.9999 (Kỳ vọng: 9)
C - D = 1.0000 (Kỳ vọng: 1)
A + C = 8.0000 (Kỳ vọng: 8)
B - D = 2.0000 (Kỳ vọng: 2)
```

Nhận xét: Hệ phương trình này có vô số nghiệm vì các phương trình phụ thuộc tuyến tính vào nhau.

BÀI 11:

YÊU CẦU 1:

```
import torch
X = torch.arange(24)
print("Tensor X gốc (24 phần tử):\n", X)

print("\n--- Reshape thành 4x6 ---")
print(X.reshape(4, 6))

print("\n--- Reshape thành 2x12 ---")
print(X.reshape(2, 12))

print("\n--- Reshape thành 2x3x4 ---")
print(X.reshape(2, 3, 4))
```

```
Tensor X gốc (24 phần tử):
 tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
        18, 19, 20, 21, 22, 23])

--- Reshape thành 4x6 ---
tensor([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11],
        [12, 13, 14, 15, 16, 17],
        [18, 19, 20, 21, 22, 23]])

--- Reshape thành 2x12 ---
tensor([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11],
        [12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23]])

--- Reshape thành 2x3x4 ---
tensor([[[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]],

        [[12, 13, 14, 15],
         [16, 17, 18, 19],
         [20, 21, 22, 23]]])
```

YÊU CẦU 2:

```
A = torch.randn(3, 4)
B = torch.randn(3, 4)

print("A + B:\n", A + B)
print("\nA * B (Nhân từng phần tử):\n", A * B)
print("\nA / B:\n", A / B)

print("\nTrung bình theo cột của A:\n", torch.mean(A, dim=0))
print("\nĐộ lệch chuẩn theo cột của A:\n", torch.std(A, dim=0))
```

```
A + B:
 tensor([[ 3.0097, -0.7747, -1.7722, -0.9695],
        [ 1.1292,  1.0626, -1.6924,  0.1769],
        [-0.9883, -1.1534,  0.2983, -1.3418]])

A * B (Nhân từng phần tử):
 tensor([[ 1.2925,  0.1473,  0.7791,  0.0241],
        [ 0.0757, -0.1127,  0.6433, -0.0357],
        [-0.5298,  0.2770,  0.0173,  0.4360]])

A / B:
 tensor([[ 0.2083,  0.7626,  1.1930, 37.0311],
        [ 0.0676, -0.0838,  0.5167, -0.4046],
        [-3.5632,  0.4195,  2.7777,  1.4305]])

Trung bình theo cột của A:
 tensor([-0.2612, -0.2577, -0.4404, -0.6180])

Độ lệch chuẩn theo cột của A:
 tensor([0.9893, 0.1391, 0.6033, 0.4379])
```

YÊU CẦU 3:

```
x = torch.tensor(2.0, requires_grad=True)
y = 2*x**2 + 5*x + 3
y.backward()
print("Giá trị đạo hàm dy/dx tại x=2 do PyTorch tính:", x.grad.item())
```

```
Giá trị đạo hàm dy/dx tại x=2 do PyTorch tính: 13.0
```

YÊU CẦU 4:

```
import numpy as np

polynomial = np.poly1d([2, -3, 5])
N = 20
X_data = np.random.randn(N, 1) * 5
Y_data = polynomial(X_data)
XX = np.hstack([X_data * X_data, X_data, np.ones_like(X_data)])
x_tensor = torch.tensor(XX, dtype=torch.float32)
y_tensor = torch.tensor(Y_data, dtype=torch.float32)
w = torch.randn(3, 1, requires_grad=True, dtype=torch.float32)
optimizer = torch.optim.NAdam([w], lr=0.01)

for epoch in range(3000):
    optimizer.zero_grad()
    y_pred = x_tensor @ w
    mse = torch.mean(torch.square(y_tensor - y_pred))
    mse.backward()
    optimizer.step()

print("Các hệ số học được sau 3000 vòng lặp:")
print(w.detach().numpy())
```

```
Các hệ số học được sau 3000 vòng lặp:
[[ 2.0011265]
 [-2.9991639]
 [ 4.9460373]]
```

KẾT THÚC